

# Algorithm Analysis Fundamentals

*Time Complexity Analysis of Iterative and Recursive Algorithms*

Laboratory Record

Name: **J Karthik**

Register Number: **192472136**

# Algorithm Analysis Fundamentals

*Time complexity analysis of iterative and recursive algorithms; verification of Big-O, Omega and Theta notations for sample programs.*

## Q1. Linear Search (Iterative)

---

### Aim

To search for a given key in an array using linear (sequential) search and to analyze its time complexity.

### Algorithm

1. Start.
2. Read the array and the key element to be searched.
3. Set  $i = 0$ .
4. Repeat while  $i < n$ : if  $\text{array}[i]$  equals key, return index  $i$ .
5. Increment  $i$  and continue scanning.
6. If the end of the array is reached without a match, return -1 (not found).
7. Display the result.
8. Stop.

### Python Program

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30
result = linear_search(arr, key)
if result != -1:
    print(f"Key found at index {result}")
else:
    print("Key not found")
```

### Input

```
arr = [10, 25, 30, 45, 50]
key = 30
```

### Output

Key found at index 2

### Result

The program correctly searches the key using linear search. Complexity verified:  $\Omega(1)$  best case (key at index 0),  $O(n)$  worst case (key at end / absent),  $\Theta(n)$  average case, since every unmatched element requires one comparison.

## Q2. Binary Search (Recursive)

---

### Aim

To search a key in a sorted array using recursive binary search and to compare its complexity with linear search.

### Algorithm

9. Start.
10. Read the sorted array and the key.
11. Define `binary_search(arr, low, high, key)`.
12. If `low > high`, return -1 (not found).
13. Compute `mid = (low + high) // 2`.
14. If `arr[mid]` equals key, return mid.
15. If `arr[mid] < key`, recursively search the right half (`mid+1` to `high`).
16. Else recursively search the left half (`low` to `mid-1`).
17. Display the result.
18. Stop.

### Python Program

```
def binary_search(arr, low, high, key):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == key:
        return mid
    elif arr[mid] < key:
        return binary_search(arr, mid + 1, high, key)
    else:
        return binary_search(arr, low, mid - 1, key)

arr = [5, 10, 15, 20, 25]
key = 20
result = binary_search(arr, 0, len(arr) - 1, key)
if result != -1:
    print(f"Key found at index {result}")
else:
    print("Key not found")
```

### Input

```
arr = [5, 10, 15, 20, 25]
key = 20
```

### Output

Key found at index 3

### Result

Binary search on a sorted array of size  $n$  runs in  $\Omega(1)$  best case,  $O(\log n)$  worst case and  $\Theta(\log n)$  average case, confirming the logarithmic efficiency of the divide-and-conquer approach compared to linear search's  $O(n)$ .

### Q3. Factorial (Iterative vs Recursive)

---

#### Aim

To compute the factorial of a number using both iterative and recursive methods and to compare their time and space behaviour.

#### Algorithm

19. Start.
20. Read n.
21. Iterative method: initialize fact = 1; for i = 1 to n, set fact = fact \* i.
22. Recursive method: define factorial(n); if n == 0 return 1; else return n \* factorial(n-1).
23. Display results of both methods.
24. Stop.

#### Python Program

```
def factorial_iterative(n):
    fact = 1
    for i in range(1, n + 1):
        fact *= i
    return fact

def factorial_recursive(n):
    if n == 0:
        return 1
    return n * factorial_recursive(n - 1)

n = 5
print("Iterative:", factorial_iterative(n))
print("Recursive:", factorial_recursive(n))
```

#### Input

n = 5

#### Output

Iterative: 120  
Recursive: 120

#### Result

Both approaches run in  $O(n)$  time since each performs  $n$  multiplications. The iterative version uses  $O(1)$  auxiliary space, while the recursive version uses  $O(n)$  stack space due to  $n$  nested calls.

## Q4. Fibonacci Series

---

### Aim

To generate the first n Fibonacci numbers using iterative, naive recursive and memoized recursive methods, and to compare their complexities.

### Algorithm

25. Start.
26. Read n.
27. Iterative method: maintain two variables a=0, b=1; repeat n times, append a, update a,b = b, a+b.
28. Naive recursive method:  $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ , with base cases  $\text{fib}(0)=0$ ,  $\text{fib}(1)=1$ .
29. Memoized method: store already computed  $\text{fib}(n)$  values in a dictionary to avoid recomputation.
30. Display the sequence generated by all three methods.
31. Stop.

### Python Program

```
def fib_iterative(n):
    a, b = 0, 1
    seq = []
    for _ in range(n):
        seq.append(a)
        a, b = b, a + b
    return seq

def fib_recursive(n):
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)

def fib_memo(n, memo={}):
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

n = 6
print("Iterative:", fib_iterative(n))
print("Recursive (naive):", [fib_recursive(i) for i in range(n)])
print("Recursive (memoized):", [fib_memo(i) for i in range(n)])
```

### Input

n = 6

### Output

Iterative: [0, 1, 1, 2, 3, 5]  
Recursive (naive): [0, 1, 1, 2, 3, 5]  
Recursive (memoized): [0, 1, 1, 2, 3, 5]

### Result

All three methods produce identical output. The iterative method runs in  $O(n)$ , the naive recursive method in  $O(2^n)$  due to repeated subproblem evaluation, and the memoized recursive method in  $O(n)$  since each subproblem is solved only once.

## Q5. Matrix Multiplication

---

### Aim

To multiply two matrices using nested loops and to analyze the resulting time complexity.

### Algorithm

32. Start.
33. Read matrix A of size p x q and matrix B of size q x r.
34. Initialize result matrix C of size p x r with zeros.
35. For i = 0 to p-1: for j = 0 to r-1: for k = 0 to q-1:  $C[i][j] += A[i][k] * B[k][j]$ .
36. Display the resultant matrix C.
37. Stop.

### Python Program

```
def matrix_multiply(A, B):  
    p, q = len(A), len(A[0])  
    r = len(B[0])  
    C = [[0] * r for _ in range(p)]  
    for i in range(p):  
        for j in range(r):  
            for k in range(q):  
                C[i][j] += A[i][k] * B[k][j]  
    return C
```

```
A = [[1, 2], [3, 4]]  
B = [[5, 6], [7, 8]]  
print(matrix_multiply(A, B))
```

### Input

```
A = [[1, 2], [3, 4]]  
B = [[5, 6], [7, 8]]
```

### Output

```
[[19, 22], [43, 50]]
```

### Result

Three nested loops each of size n give a time complexity of  $O(n^3)$  for n x n matrices, demonstrating cubic growth as the matrix dimension increases.

## Q6. Merge Sort

---

### Aim

To sort an array using the merge sort divide-and-conquer technique and to analyze its guaranteed time complexity.

### Algorithm

38. Start.
39. If the array has one or zero elements, return it as already sorted.
40. Divide the array into a left half and a right half.
41. Recursively apply merge sort on the left half.
42. Recursively apply merge sort on the right half.
43. Merge the two sorted halves into a single sorted array.
44. Stop.

### Python Program

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))

arr = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(arr))
```

### Input

arr = [38, 27, 43, 3, 9, 82, 10]

### Output

[3, 9, 10, 27, 38, 43, 82]

### Result

Merge sort always divides the array in half and merges in linear time, giving  $O(n \log n)$  time complexity in the best, average and worst cases, regardless of the initial arrangement of the input.

## Q7. Quick Sort

---

### Aim

To sort an array using recursive quick sort and to analyze how the choice of pivot affects performance.

### Algorithm

45. Start.
46. If the array has one or zero elements, return it as already sorted.
47. Choose a pivot element (last element of the array).
48. Partition remaining elements into a 'left' list ( $< \text{pivot}$ ) and a 'right' list ( $\geq \text{pivot}$ ).
49. Recursively quick sort the left list.
50. Recursively quick sort the right list.
51. Concatenate left result, pivot and right result.
52. Stop.

### Python Program

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[-1]  
    left = [x for x in arr[:-1] if x < pivot]  
    right = [x for x in arr[:-1] if x >= pivot]  
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
arr = [10, 7, 8, 9, 1, 5]  
print(quick_sort(arr))
```

### Input

```
arr = [10, 7, 8, 9, 1, 5]
```

### Output

```
[1, 5, 7, 8, 9, 10]
```

### Result

Quick sort gives  $\Omega(n \log n)$  with a balanced pivot,  $O(n^2)$  worst case when the pivot repeatedly splits the array unevenly (e.g. already sorted input with last-element pivot), and  $\Theta(n \log n)$  on average, showing pivot choice is critical to performance.

## Q8. Tower of Hanoi

---

### Aim

To solve the Tower of Hanoi problem for n disks using recursion and to analyze its exponential growth.

### Algorithm

53. Start.
54. Read the number of disks n, and the source, auxiliary and destination pegs.
55. If  $n == 1$ , move the single disk directly from source to destination.
56. Else, recursively move the top n-1 disks from source to auxiliary (using destination as helper).
57. Move the remaining (largest) disk from source to destination.
58. Recursively move the n-1 disks from auxiliary to destination (using source as helper).
59. Stop.

### Python Program

```
def tower_of_hanoi(n, source, auxiliary, destination):  
    if n == 1:  
        print(f"Move disk 1 from {source} to {destination}")  
        return  
    tower_of_hanoi(n - 1, source, destination, auxiliary)  
    print(f"Move disk {n} from {source} to {destination}")  
    tower_of_hanoi(n - 1, auxiliary, source, destination)  
  
tower_of_hanoi(3, 'A', 'B', 'C')
```

### Input

n = 3, source='A', auxiliary='B', destination='C'

### Output

```
Move disk 1 from A to C  
Move disk 2 from A to B  
Move disk 1 from C to B  
Move disk 3 from A to C  
Move disk 1 from B to A  
Move disk 2 from B to C  
Move disk 1 from A to C
```

### Result

The total number of moves for n disks is  $2^n - 1$  (7 moves for n=3, verified above), which confirms the exponential time complexity  $O(2^n)$  of the recursive solution.

## Q9. GCD (Euclidean Algorithm)

---

### Aim

To compute the Greatest Common Divisor (GCD) of two numbers using the recursive Euclidean algorithm and to analyze its logarithmic complexity.

### Algorithm

60. Start.
61. Read two integers a and b.
62. Define gcd(a, b): if  $b == 0$ , return a.
63. Else, return gcd(b, a mod b).
64. Display the result.
65. Stop.

### Python Program

```
def gcd(a, b):  
    if b == 0:  
        return a  
    return gcd(b, a % b)
```

```
a, b = 48, 18  
print("GCD:", gcd(a, b))
```

### Input

a = 48, b = 18

### Output

GCD: 6

### Result

The Euclidean algorithm reduces the problem size by at least half every two recursive calls (via the modulo operation), giving a time complexity of  $O(\log(\min(a, b)))$ , which is verified to be far faster than a brute-force divisor search.

## Q10. Insertion Sort

---

### Aim

To sort an array using insertion sort and to analyze its sensitivity to the initial ordering of the input.

### Algorithm

66. Start.
67. Read the array of size  $n$ .
68. For  $i = 1$  to  $n-1$ : set  $key = arr[i]$  and  $j = i-1$ .
69. While  $j \geq 0$  and  $arr[j] > key$ : shift  $arr[j]$  to  $arr[j+1]$  and decrement  $j$ .
70. Place  $key$  at position  $j+1$ .
71. Repeat until the whole array is processed.
72. Stop.

### Python Program

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

```
arr = [12, 11, 13, 5, 6]
print(insertion_sort(arr))
```

### Input

```
arr = [12, 11, 13, 5, 6]
```

### Output

```
[5, 6, 11, 12, 13]
```

### Result

Insertion sort gives  $\Omega(n)$  on an already-sorted input (inner while loop never executes),  $O(n^2)$  on a reverse-sorted input (maximum shifting), and  $\Theta(n^2)$  on average, demonstrating strong input sensitivity.

## Q11. Heap Sort

---

### Aim

To sort an array using heap sort and to verify its consistent performance irrespective of input arrangement.

### Algorithm

73. Start.
74. Read the array of size  $n$ .
75. Build a max heap from the array (heapify from the last non-leaf node up to the root).
76. Repeat for  $i = n-1$  down to 1: swap the root (maximum) with element at index  $i$ .
77. Reduce the heap size by 1 and heapify the root of the reduced heap.
78. Continue until the heap size is 1; the array is now sorted.
79. Stop.

### Python Program

```
def heapify(arr, n, i):
    largest = i
    l, r = 2 * i + 1, 2 * i + 2
    if l < n and arr[l] > arr[largest]:
        largest = l
    if r < n and arr[r] > arr[largest]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr

arr = [4, 10, 3, 5, 1]
print(heap_sort(arr))
```

### Input

arr = [4, 10, 3, 5, 1]

### Output

[1, 3, 4, 5, 10]

### Result

Heap sort performs heap construction in  $O(n)$  followed by  $n$  extractions each costing  $O(\log n)$ , giving an overall time complexity of  $O(n \log n)$  in the best, average and worst cases, unlike quick sort or insertion sort whose worst case degrades.

## Q12. Counting Inversions

---

### Aim

To count the number of inversions in an array using a modified merge sort and to analyze its complexity.

### Algorithm

80. Start.
81. If the array has one or zero elements, return it with 0 inversions.
82. Split the array into left and right halves.
83. Recursively count inversions within the left half and within the right half.
84. Merge the two sorted halves; whenever an element from the right half is placed before remaining elements of the left half, add the count of those remaining left elements as split inversions.
85. Sum left inversions, right inversions and split inversions.
86. Stop.

### Python Program

```
def merge_count(arr):
    if len(arr) <= 1:
        return arr, 0
    mid = len(arr) // 2
    left, inv_left = merge_count(arr[:mid])
    right, inv_right = merge_count(arr[mid:])
    merged = []
    i = j = inv_split = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
            inv_split += len(left) - i
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged, inv_left + inv_right + inv_split

arr = [2, 4, 1, 3, 5]
sorted_arr, inversions = merge_count(arr)
print("Sorted array:", sorted_arr)
print("Number of inversions:", inversions)
```

### Input

```
arr = [2, 4, 1, 3, 5]
```

### Output

```
Sorted array: [1, 2, 3, 4, 5]
```

```
Number of inversions: 3
```

### Result

The modified merge sort counts inversions while merging, at no extra asymptotic cost, giving  $O(n \log n)$  time complexity - far better than the brute-force  $O(n^2)$  pairwise comparison approach.

## Q13. Maximum Subarray Sum

---

### Aim

To find the maximum subarray sum using Kadane's algorithm and the divide-and-conquer method, and to compare their efficiencies.

### Algorithm

87. Start.
88. Kadane's method: initialize `cur_sum = max_sum = arr[0]`; for each subsequent element `x`, set `cur_sum = max(x, cur_sum + x)` and update `max_sum`.
89. Divide-and-conquer method: split the array into left and right halves.
90. Recursively find the maximum subarray sum in the left half and in the right half.
91. Compute the maximum sum of a subarray crossing the midpoint.
92. Return the maximum of the left, right and crossing sums.
93. Compare the results and running times of both methods.
94. Stop.

### Python Program

```
def kadane(arr):
    max_sum = cur_sum = arr[0]
    for x in arr[1:]:
        cur_sum = max(x, cur_sum + x)
        max_sum = max(max_sum, cur_sum)
    return max_sum

def max_crossing_sum(arr, low, mid, high):
    left_sum, total = float('-inf'), 0
    for i in range(mid, low - 1, -1):
        total += arr[i]
        left_sum = max(left_sum, total)
    right_sum, total = float('-inf'), 0
    for i in range(mid + 1, high + 1):
        total += arr[i]
        right_sum = max(right_sum, total)
    return left_sum + right_sum

def max_subarray_dc(arr, low, high):
    if low == high:
        return arr[low]
    mid = (low + high) // 2
    return max(max_subarray_dc(arr, low, mid),
               max_subarray_dc(arr, mid + 1, high),
               max_crossing_sum(arr, low, mid, high))

arr = [-2, -3, 4, -1, -2, 1, 5, -3]
print("Kadane's:", kadane(arr))
print("Divide & Conquer:", max_subarray_dc(arr, 0, len(arr) - 1))
```

### Input

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
```

### Output

Kadane's: 7

Divide & Conquer: 7

## Result

Both methods return the same maximum subarray sum of 7 (subarray [4, -1, -2, 1, 5]). Kadane's algorithm runs in  $O(n)$  with a single pass, while the divide-and-conquer method takes  $O(n \log n)$ , so Kadane's is the more efficient choice.

## Q14. Exponentiation

---

### Aim

To compute  $x$  raised to the power  $n$  using iterative and recursive fast-power methods and to compare their complexities.

### Algorithm

95. Start.
96. Read  $x$  and  $n$ .
97. Iterative method: initialize  $result = 1$ ; multiply  $result$  by  $x$ ,  $n$  times.
98. Recursive fast power method: if  $n == 0$ , return 1.
99. Compute  $half = power(x, n // 2)$ .
100. If  $n$  is even, return  $half * half$ ; else return  $x * half * half$ .
101. Display both results.
102. Stop.

### Python Program

```
def power_iterative(x, n):  
    result = 1  
    for _ in range(n):  
        result *= x  
    return result  
  
def power_fast(x, n):  
    if n == 0:  
        return 1  
    half = power_fast(x, n // 2)  
    if n % 2 == 0:  
        return half * half  
    return x * half * half  
  
x, n = 2, 10  
print("Iterative:", power_iterative(x, n))  
print("Recursive (fast power):", power_fast(x, n))
```

### Input

$x = 2, n = 10$

### Output

Iterative: 1024  
Recursive (fast power): 1024

### Result

The iterative method performs  $n$  multiplications giving  $O(n)$  time, while the recursive fast-power method halves  $n$  at every call, giving  $O(\log n)$  time - demonstrating how a divide-and-conquer recursive formulation improves efficiency.

## Q15. Closest Pair of Points

---

### Aim

To find the closest pair of points among a given set of 2-D points using the brute-force approach and to compare it with the divide-and-conquer approach.

### Algorithm

103. Start.
104. Read the list of  $n$  points.
105. Brute-force method: for every pair of points  $(i, j)$  with  $i < j$ , compute the Euclidean distance.
106. Track the pair with the minimum distance found so far.
107. After checking all  $C(n, 2)$  pairs, report the closest pair and its distance.
108. (Divide-and-conquer method: sort points, split into halves, recursively solve each half, then check a narrow strip near the dividing line for a closer cross pair.)
109. Stop.

### Python Program

```
import math

def distance(p1, p2):
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)

def closest_pair_brute(points):
    n = len(points)
    min_dist = float('inf')
    pair = None
    for i in range(n):
        for j in range(i + 1, n):
            d = distance(points[i], points[j])
            if d < min_dist:
                min_dist = d
                pair = (points[i], points[j])
    return pair, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
pair, dist = closest_pair_brute(points)
print(f'Closest pair {pair}, Distance = {dist:.4f}')
```

### Input

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

### Output

```
Closest pair ((1, 2), (3, 1)), Distance = 2.2361
```

### Result

The brute-force approach checks all  $n(n-1)/2$  pairs, giving  $O(n^2)$  time. The optimized divide-and-conquer approach sorts the points and limits the strip check, achieving  $O(n \log n)$  - demonstrating the benefit of divide-and-conquer for large point sets.